

JOBEASE

Technical Project Documentation

Full-Stack AI-Powered Job Portal & Resume Builder

Capstone Engineering Project

Nitesh | Final Year B.E.

Deployed: jobease-1.onrender.com

2024–2025

1. Project Overview

JobEase is a production-deployed, full-stack web application that serves as both a job portal and an AI-enhanced resume builder. It was designed and built as a final-year engineering capstone project, with the goal of solving a real-world problem: the friction job seekers face when applying for jobs and building professional resumes.

The application supports two distinct user roles — job seekers and recruiters — with separate dashboards, workflows, and protected routes. It was validated in a real-world setting at a college job fair, where actual users registered, created profiles, and applied for jobs through the live platform.

Live Deployment

Frontend: <https://jobease-1.onrender.com>

Backend: <https://jobease.onrender.com>

Status: Production — Deployed on Render (Free Tier)

1.1 Project Goals

- Build a fully functional, deployed job portal from scratch using the MERN stack
- Integrate AI (Google Gemini) to provide real resume enhancement capabilities
- Implement a multi-template resume builder with PDF export functionality
- Handle real user authentication with Clerk, including role-based access control
- Gain hands-on experience with deployment, debugging, and production-level issues
- Demonstrate architectural decision-making during capstone examination

1.2 Real-World Validation

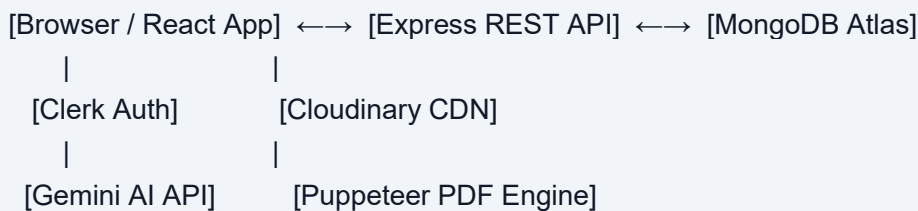
JobEase was used at an actual college job fair. Real students registered as job seekers, companies registered as recruiters, and live job postings were made and applied to during the event. This validated both the core job portal functionality and the authentication flow under real concurrent usage.

2. System Architecture

JobEase follows a classic client-server architecture with a clear separation between frontend, backend, and external services. The system is designed as a monorepo with separate client/ and server/ directories deployed independently on Render.

2.1 High-Level Architecture

Architecture Overview



2.2 Frontend Architecture (React / Vite)

The frontend is a Single Page Application built with React 18 and bundled with Vite. It uses React Router v6 for client-side routing and Tailwind CSS for utility-first styling.

Component Structure

- pages/ — Top-level route components (HomePage, JobListings, ResumeBuilder, RecruiterDashboard, etc.)
- components/ — Reusable UI components (JobCard, Navbar, TemplatePreview, etc.)
- context/ — React Context providers for global state (AuthContext, ResumeContext)
- api/ — Axios-based API call functions
- utils/ — Helper functions and constants

State Management

Global state is managed through React Context API. The application does not use Redux — a deliberate choice to keep the architecture simple for a project of this scale. Key contexts include authentication state (syncd with Clerk) and resume builder state.

2.3 Backend Architecture (Node.js / Express)

The backend is a RESTful API built with Express.js. It handles business logic, database interactions, file processing (PDF generation), and proxies AI requests to Gemini.

Route Structure

Route	Responsibility
/api/auth	Clerk webhook sync, user creation/update
/api/jobs	CRUD for job listings, filtering, pagination
/api/applications	Job application submission and status tracking
/api/companies	Company profile management, logo upload
/api/resume	Resume data CRUD, PDF generation endpoint
/api/ai	Gemini AI enhancement proxy endpoint

2.4 Database Architecture (MongoDB)

MongoDB Atlas is used as the primary database, accessed via Mongoose ODM. The schema design is document-oriented, which fits naturally with the resume builder data (nested arrays of experience, education, skills) and job listing data.

Core Collections

Collection	Key Fields
users	Clerk userId, role (seeker/recruiter), profile data, resume reference
jobs	Title, description, salary, location, companyId, status, applicants[]
companies	Name, logo (Cloudinary URL), description, recruiterId
applications	jobId, userId, status, appliedAt, coverLetter
resumes	userId, templateId, personal info, experience[], education[], skills, colors

Note: MongoDB does not support cascading deletes natively. When a company is deleted, associated jobs must be explicitly removed via `Job.deleteMany({ companyId })`. This was a bug discovered in production and subsequently fixed.

3. Technology Stack

3.1 Frontend

Technology	Role
React 18	Core UI library — component-based architecture, hooks
Vite	Build tool and dev server — significantly faster than CRA
React Router v6	Client-side routing with protected routes and nested layouts
Tailwind CSS	Utility-first CSS framework — rapid UI development
Clerk (React SDK)	Authentication — sign-in, sign-up, user management UI
Axios	HTTP client for REST API calls
React Toastify	Toast notification system for user feedback
Lucide React	Icon library — clean, consistent SVG icons

3.2 Backend

Technology	Role
Node.js	JavaScript runtime environment
Express.js	Web framework — routing, middleware, error handling
Mongoose	MongoDB ODM — schema definition, validation, queries
Clerk (Express SDK)	Server-side auth middleware — JWT verification
Puppeteer	Headless Chrome — generates PDF resumes from HTML
Google Gemini AI	AI model — resume bullet point and summary enhancement
Cloudinary	Cloud image storage and CDN for company logos
Multer	Multipart form data middleware for file uploads
CORS	Cross-origin resource sharing configuration
dotenv	Environment variable management

3.3 Database & Infrastructure

Service	Role
---------	------

MongoDB Atlas	Cloud-hosted MongoDB cluster (M0 free tier)
Render	Hosting platform — frontend (static site) + backend (web service)
Cloudinary CDN	Image delivery and transformation

3.4 Development Tools

Tool	Purpose
MongoDB Compass	GUI for inspecting and querying MongoDB collections
Mongosh	MongoDB shell for running manual queries and fixes
Postman	API testing and endpoint verification
Git / GitHub	Version control
VS Code	Primary IDE

4. Features

4.1 Job Seeker Features

Job Portal

- Browse all active job listings with real-time search and filters
- Filter by job title, location, salary range, and job type
- Paginated job listing page with smooth navigation
- One-click job application with optional cover letter
- Application tracking — view status of all submitted applications
- Company profile pages with logo, description, and open roles

Resume Builder

- Five distinct resume templates: Classic, Modern, Minimal, Minimal-Image, and ATS Friendly
- Real-time live preview — changes reflect instantly as you type
- Font family selector and custom color pickers (primary and accent colors)
- Structured sections: Personal Info, Summary, Experience, Education, Skills, Projects, Certifications
- Gemini AI integration — click to enhance any bullet point or summary with AI suggestions
- One-click PDF export powered by Puppeteer (server-side rendering)
- Resume data persisted to MongoDB — auto-saves and loads on return visits
- Template switching with automatic data format conversion

4.2 Recruiter Features

- Company registration with logo upload (stored on Cloudinary)
- Create, edit, and delete job postings
- Recruiter dashboard — overview of all posted jobs and applicant counts
- View all applications per job listing with applicant details
- Update application status (Pending, Reviewed, Shortlisted, Rejected)

4.3 Authentication & Security

- Clerk-powered authentication — social login (Google, GitHub) and email/password
- Role-based access control — different dashboards and route protection per role

- Protected routes with redirect logic using Clerk isLoaded + isSignedIn pattern
- Server-side route protection via Clerk Express middleware (requireAuth)
- userId passed in request headers, verified against Clerk JWT on sensitive endpoints

4.4 Resume Templates

Template	Description
Classic	Traditional two-column layout, black/white, high ATS compatibility
Modern	Sidebar design with accent color, visual section dividers
Minimal	Clean single-column layout, typography-focused, no skills icons
Minimal-Image	Minimal layout with circular profile photo in header
ATS Friendly	Plain text-optimized, no columns, no icons — pure ATS parsing target

5. Problems Faced & Solutions

This section documents the significant technical challenges encountered during development, the root cause analysis, and the solutions applied. These were real production bugs, not hypothetical issues.

Problem 1: Clerk Authentication Race Condition

Bug

Users were being redirected to /sign-in immediately on page load, even when they were already logged in. The app seemed to "forget" the signed-in state on every refresh.

Root Cause

Clerk's `useAuth()` hook returns `isSignedIn` as `undefined` before the SDK has finished initializing. The app was checking `if (!isSignedIn)` which evaluates to `true` when the value is `undefined` — triggering a premature redirect before Clerk even confirmed the user's state.

Solution

```
if (!isLoading) return <LoadingSpinner />;  
if (!isSignedIn) return <Navigate to="/sign-in" />;
```

Added an `isLoading` guard before any auth-gated logic. The component renders a loading state until Clerk confirms initialization, then checks the actual signed-in status.

Problem 2: Puppeteer Not Applying Tailwind CSS

Bug

Generated PDFs showed unstyled HTML — no colors, no fonts, no layout. All Tailwind classes were ignored. The resume looked like raw 1990s HTML.

Root Cause

Puppeteer receives a raw HTML string and renders it in an isolated headless browser context. It has no access to the Vite build pipeline or the compiled Tailwind CSS file. Tailwind's utility classes only exist after the PostCSS build process generates the stylesheet — Puppeteer never sees that file.

Solution

```
const html = `  
  <html><head>  
    <script src="https://cdn.tailwindcss.com"></script>  
    <link href="https://fonts.googleapis.com/css2?family=Inter..."  
rel="stylesheet">  
  </head><body>${resumeHTML}</body></html>`  
  
await page.setContent(html, { waitUntil: "networkidle0" });  
await page.waitForTimeout(800); // settle delay for font render
```

Inject the Tailwind CDN script and Google Fonts directly into the HTML string. Use `waitUntil: networkidle0` so Puppeteer waits for all external resources to load before capturing the PDF.

Problem 3: Orphaned Job Cards After Company Deletion

Bug

When a recruiter deleted their company, the company was removed from the database, but all associated job listings remained. These jobs showed up on the public board with broken/null company information.

Root Cause

MongoDB has no built-in cascading delete mechanism. Mongoose schema references (`companyId` in `Job`) do not automatically cascade when a referenced document is deleted — this must be handled manually in application code.

Solution

```
// In the DELETE /api/companies/:id controller:  
await Company.findByIdAndDelete(companyId);  
await Job.deleteMany({ companyId: companyId }); // explicit cascade
```

Added explicit cascade deletion in the company delete controller. Additionally added null checks on `.populate()` results in the job listing API to handle any orphaned records that existed before the fix.

Problem 4: Template Switching Skills Format Mismatch

Bug

Switching from the ATS Friendly template to Modern (or vice versa) caused the skills section to either crash or display incorrectly. Skills data would either be missing or show `[object Object]`.

Root Cause

The ATS Friendly template stores skills as a flat array of strings (`["React", "Node.js"]`) for maximum compatibility. The Modern and Minimal templates use a categorized object format (`{ category: "Frontend", skills: ["React", "CSS"] }`). When template switching, the data was passed directly without format conversion.

Solution

```
function handleTemplateChange(newTemplate) {
  const currentSkills = resumeData.skills;
  let convertedSkills = currentSkills;
  if (newTemplate === "ats" && typeof currentSkills[0] === "object") {
    convertedSkills = currentSkills.flatMap(cat => cat.skills);
  } else if (newTemplate !== "ats" && typeof currentSkills[0] === "string") {
    convertedSkills = [{ category: "Skills", skills: currentSkills }];
  }
  setResumeData({ ...resumeData, skills: convertedSkills, template: newTemplate });
}
```

Problem 5: SPA Routing Failures on Render (404 on Refresh)

Bug

Navigating directly to a URL like `/jobs/123` or refreshing on any route other than `/` returned a 404 error page from Render.

Root Cause

React Router handles routing on the client side — the server only has one file: index.html. When a user directly accesses /jobs/123, Render's static file server looks for a file at that path, finds nothing, and returns 404. Client-side routing only works once index.html has loaded and React has bootstrapped.

Solution

Created a _redirects file in client/public/ with a single rule:

```
/* /index.html 200
```

This tells Render to serve index.html for all paths with a 200 status, allowing React Router to handle the routing client-side after the app loads.

Problem 6: Stale MongoDB Data from Schema Migrations

Bug

After updating Mongoose schemas (adding new required fields, changing types), existing documents in MongoDB did not have the new fields, causing validation errors and undefined crashes on the frontend.

Root Cause

MongoDB is schemaless at the database level. Existing documents are not automatically updated when a Mongoose schema changes. Documents created before the schema change simply lack the new fields.

Solution

Used Mongosh to run targeted migration scripts to backfill missing fields on existing documents. Also added safe defaults in Mongoose schema definitions (default: null, default: []) to ensure backward compatibility for new optional fields.

6. Key Learnings

This section documents the most significant technical and architectural learnings from building JobEase — lessons that wouldn't appear in a tutorial but only emerge from building and debugging a real production application.

6.1 Authentication Timing is Everything

The most important thing I learned about Clerk (and third-party auth SDKs in general) is that initialization is asynchronous. The pattern:

```
const { isLoading, isSignedIn, user } = useAuth();  
if (!isLoading) return null; // ALWAYS check this first
```

This pattern applies broadly: any async initialization (auth, data fetching, context loading) requires a "ready" guard before conditional logic, or you will get false negatives from undefined values.

6.2 Build-Time vs Runtime Environment Variables

Vite bakes VITE_ prefixed variables into the JavaScript bundle at build time. This means:

- Changing a VITE_ env variable on Render requires a full redeploy (not just restart)
- Secrets should never use the VITE_ prefix — they become public in the bundle
- Server-side env variables (without VITE_) are loaded at runtime and are genuinely private

This distinction is critical for security. The Gemini API key was initially exposed via a VITE_ variable — a security issue identified and flagged for fix.

6.3 MongoDB Has No Magic Cascades

Relational databases have ON DELETE CASCADE. MongoDB does not. Every relationship in a document database must be manually managed at the application layer. This means:

- When deleting a parent document, write explicit deleteMany for all child collections
- Use .populate() carefully — it can return null if the referenced document was deleted
- Always add null-checks after populate() in API responses

6.4 Headless Rendering Environments Are Isolated

Puppeteer renders HTML in a sandbox that knows nothing about your application. No Vite, no Tailwind build pipeline, no Google Fonts preloads. Any resource your HTML needs must be either:

- Inlined directly into the HTML string (styles, fonts as base64)
- Loaded from a CDN URL via a `<script>` or `<link>` tag in the document

The `waitUntil: networkidle0` option is essential — it makes Puppeteer wait until all CDN resources have loaded before capturing the PDF, preventing blank or unstyled output.

6.5 Static Hosting and SPA Routing

Every static hosting provider needs explicit configuration to serve a Single Page Application correctly. Without the `_redirects` rule, direct URL access and page refreshes break completely. This is a one-line fix but a frustrating thing to debug for the first time, especially when everything works perfectly in local development.

6.6 Schema Design Decisions Have Long-Term Consequences

Choosing to store resume skills as `string[]` in one template and `object[]` in another created a runtime format mismatch bug. The lesson: when designing schemas, think about all the shapes the data will need to take across the entire application, not just the current use case. Conversion functions at boundaries are a code smell indicating a schema design issue.

6.7 Production Debugging is Different from Development

Several bugs only manifested in production:

- CORS errors were hidden in local dev (same origin) but broke on Render
- Render's free tier has cold starts — first request after inactivity can take 30+ seconds
- Environment variables behave differently in build vs runtime contexts
- MongoDB Atlas network access list had to be configured to allow Render's IPs

The practice of checking `X-Denied-Reason` headers and Render logs directly saved significant debugging time.

7. Strengths & Limitations

7.1 Strengths (What Works Well)

Real-World Validated

Used at an actual college job fair with real registrations, job postings, and applications — not just a demo project. This proves the core flows work under real usage conditions, not just test scenarios.

Genuine AI Integration

Gemini AI is integrated into the resume builder in a functional, user-facing way. It doesn't just generate a resume from scratch — it enhances specific bullet points and summarizes the user has already written, which is a more practical use case.

Full Deployment Pipeline

The application is not just "it works on localhost." It has a complete deployment pipeline with separate frontend and backend services, environment variable management, Cloudinary CDN integration, and MongoDB Atlas — all running in production.

End-to-End Ownership

Every layer of the stack — auth, database, API, frontend, file processing, AI — was designed and implemented by a single developer. This demonstrates full-stack depth rather than surface-level familiarity.

7.2 Known Limitations & Technical Debt

Security: Gemini API Key Exposure

The Gemini API key is currently consumed on the frontend via a `VITE_` env variable, making it visible in the JavaScript bundle. It must be moved to a backend proxy endpoint before this is considered production-secure.

No Server-Side Pagination on All Endpoints

Job listings implement pagination, but some other list endpoints return full collections. At scale, these would become performance bottlenecks.

No Search Debounce

The job search filter triggers API calls on every keystroke. This creates unnecessary load on the backend. A 300-400ms debounce should be added.

Context Architecture Could Be More Granular

The resume builder uses a monolithic context that re-renders too broadly on small state changes. Splitting into feature-scoped contexts (`ResumeMetaContext`, `ResumeContentContext`) would reduce unnecessary re-renders.

No Centralized Axios Instance

API calls use Axios directly without a centralized client. This means auth headers, base URLs, and error interceptors are duplicated across multiple files. A single `axios.create()` instance in `api/client.js` would fix this.

8. Potential Improvements

These are architectural and feature improvements identified during development but not yet implemented. They represent the next iteration of the project.

8.1 Security Fixes (High Priority)

1. Move Gemini API key to backend — create a `/api/ai/enhance` proxy endpoint
2. Verify `userId` via Clerk JWT using `getToken()` instead of trusting request headers
3. Add rate limiting (`express-rate-limit`) to the AI endpoint to prevent abuse

8.2 Architecture Improvements

4. Centralized Axios client in `api/client.js` with auth interceptor and base URL
5. Custom hooks layer — `useJobs()`, `useApplications()`, `useResume()` to abstract data fetching
6. Feature-scoped React contexts instead of one monolithic context
7. `constants/` folder for magic strings (routes, API endpoints, status values)
8. Server-side pagination on all list endpoints, not just jobs
9. Debounce on all search/filter inputs (300ms threshold)

8.3 Feature Additions

10. Email notifications — notify applicants when application status changes
11. Resume sharing — generate a public URL for a rendered resume
12. Job recommendation engine — match seeker skills to job requirements
13. Cover letter AI generation using Gemini
14. Interview scheduling — recruiter-facing calendar for shortlisted candidates

9. Project Summary

Attribute	Value
Project Type	Full-Stack Web Application (Capstone)
Duration	~8 months (2024–2025)
Developer	Nitesh (Solo)
Frontend	React 18 + Vite + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB Atlas (Mongoose)
Auth	Clerk (JWT-based, role-aware)
AI Integration	Google Gemini API
PDF Engine	Puppeteer (headless Chrome)
Image Storage	Cloudinary CDN
Hosting	Render (Frontend + Backend)
Resume Templates	5 (Classic, Modern, Minimal, Minimal-Image, ATS)
Real-World Usage	College job fair — live registrations and applications
Frontend URL	https://jobease-1.onrender.com
Backend URL	https://jobease.onrender.com

JobEase — Built end-to-end by Nitesh
Final Year B.E. Capstone Project | 2024–2025